

```

import crypto from 'node:crypto';
import { q, one } from '../db.js';
import { registerGenerated } from './registry.js';
import { resolve } from './providers.js';
import { sign } from './signing.js';

// Anything from outside becomes a package or it stops at the step that failed.
// The job row records which step, and why, so a rejection is inspectable.
const STEPS = [
  'fetch', 'analyze', 'graph', 'license', 'security', 'pin',
  'sandbox', 'capabilities', 'permissions', 'sign', 'publish',
];

// Licenses that may be redistributed inside a package without further review.
const ALLOWED = new Set(['MIT', 'Apache-2.0', 'BSD-2-Clause', 'BSD-3-Clause', 'ISC', 'MIT-0']);
const BLOCKED = new Set(['AGPL-3.0', 'AGPL-3.0-only', 'AGPL-3.0-or-later', 'SSPL-0']);

export async function importRepo({ ctx = null, source = 'github', reference, fetch }) {
  const job = await one(
    `insert into import_job (business_id, source, reference, state)
    values ($1,$2,$3,'running') returning *`,
    [ctx?.businessId ?? null, source, reference]
  );

  const findings = {};
  try {
    // 1. fetch
    const repo = await fetchImpl(reference);
    findings.fetch = { name: repo.full_name ?? reference, private: repo.private ? true : false };

    // 2. analyze
    findings.analyze = {
      language: repo.language ?? 'unknown',
      stars: repo.stargazers_count ?? null,
      archived: repo.archived ?? false,
      pushedAt: repo.pushed_at ?? null,
    };
    if (repo.archived) return await fail(job, 'analyze', 'repository is archived');

    // 3. graph - what it depends on and what it exposes
    findings.graph = { entryPoints: repo.entryPoints ?? [], dependencies: repo.dependencies ?? [] };

    // 4. license
    const license = repo.license?.spdx_id ?? repo.license ?? null;
  } catch (err) {
    return await fail(job, 'fetch', err.message);
  }
}

```

```

findings.license = { spdx: license, allowed: ALLOWED.has(license) };
if (!license || license === 'NOASSERTION') {
  return await fail(job, 'license', 'no identifiable license', findings);
}
if (BLOCKED.has(license)) {
  return await fail(job, 'license', `${license} cannot be redistributed inside`);
}
if (!ALLOWED.has(license)) {
  return await fail(job, 'license', `${license} is not on the allow list`, findings);
}

// 5. security
const vulns = repo.vulnerabilities ?? [];
const critical = vulns.filter(v => v.severity === 'critical');
findings.security = { count: vulns.length, critical: critical.length };
if (critical.length > 0) {
  return await fail(job, 'security', `${critical.length} critical advisories`);
}

// 6. pin - a moving branch is not a package
const commit = repo.commit ?? repo.default_branch_sha ?? null;
if (!commit) return await fail(job, 'pin', 'could not resolve a commit to pin');
findings.pin = { commit };

// 7. sandbox
const sandbox = await resolve({ slot: 'sandbox', businessId: ctx?.businessId });
if (sandbox) {
  const result = await sandbox.module.test({ reference, commit, repo });
  findings.sandbox = result;
  if (!result.passed) return await fail(job, 'sandbox', result.reason ?? 'sandbox failed');
} else {
  findings.sandbox = { skipped: 'no sandbox provider bound' };
}

// 8. capabilities - what the platform will let it expose
const proposed = repo.proposedManifest;
if (!proposed) return await fail(job, 'capabilities', 'nothing declared a manifest');
findings.capabilities = { declared: proposed.capabilities ?? [] };

// 9. permissions - namespace enforcement happens here, before it exists
for (const cap of proposed.capabilities ?? []) {
  if (!cap.startsWith(proposed.key + '.')) {
    return await fail(job, 'permissions', `declares capability outside its namespace`);
  }
}
for (const [table, def] of Object.entries(proposed.schema ?? {})) {
  if ('business_id' in (def.fields ?? {})) {

```

```

        return await fail(job, 'permissions', `table ${table} tries to own busine
    }
}
findings.permissions = { ok: true };

// 10. sign
const contentHash = crypto.createHash('sha256').update(JSON.stringify(propose
await sign({
    packageKey: proposed.key, version: proposed.version, contentHash,
    signer: `import:${source}:${reference}@${commit.slice(0, 7)}\`,
    trustTier: 'community',
});
findings.sign = { contentHash };

// 11. publish
registerGenerated(proposed);
await one(
    `insert into package (key, version, kind, name, summary, manifest, source,
    values ($1,$2,$3,$4,$5,$6::jsonb,$7,$8)
    on conflict (key, version) do update set manifest = excluded.manifest retu
[proposed.key, proposed.version, proposed.kind, proposed.name, proposed.sum
    JSON.stringify(proposed), `${source}:${reference}`, contentHash]
);

await q(
    `update import_job set state = 'published', pinned_commit = $1, findings =
        produced_package = $3, produced_version = $4, finished_at = now()
        where id = $5`,
    [commit, JSON.stringify(findings), proposed.key, proposed.version, job.id]
);

return { ok: true, jobId: job.id, packageKey: proposed.key, version: proposed
} catch (e) {
    return fail(job, 'fetch', e.message, findings);
}
}

async function fail(job, step, reason, findings) {
    await q(
        `update import_job set state = 'rejected', failed_at_step = $1, reason = $2,
            findings = $3::jsonb, finished_at = now() where id = $4`,
        [step, reason, JSON.stringify(findings), job.id]
    );
    return { ok: false, jobId: job.id, failedAt: step, reason, findings };
}

async function defaultFetch(reference) {

```

```

const res = await fetch(`https://api.github.com/repos/${reference}`, {
  headers: { 'accept': 'application/vnd.github+json', 'user-agent': 'ghost-imp'
});
if (!res.ok) throw new Error(`github returned ${res.status} for ${reference}`);
return res.json();
}

export const steps = STEPS;

export async function jobs(businessId = null, limit = 50) {
  return q(
    `select source, reference, state, failed_at_step, reason, produced_package, c
      from import_job
      where ($1::uuid is null or business_id = $1)
      order by created_at desc limit ${Number(limit)}`,
    [businessId]
  );
}

```